

Bitácora de Aprendizaje

Pasaje de Lindenmayer a Gramática Libre de Contexto

Al intentar modelar la estructura fractal de los vasos sanguíneos, nos topamos con un choque teórico. Los Sistemas de Lindenmayer puros están diseñados para simular el crecimiento biológico mediante un proceso de sustitución en paralelo; es decir, todas las variables de una cadena mutan simultáneamente en un mismo paso de tiempo.

Sin embargo, los algoritmos tradicionales de análisis sintáctico (como un Parser Descendente Determinista) operan de forma estrictamente secuencial, procesando los tokens uno detrás de otro de izquierda a derecha. Un parser estándar no puede manejar mutaciones masivas y simultáneas en paralelo directamente desde su cinta de opciones.

Por ello decidimos mantener la semántica visual y fractal característica de los Sistemas de Lindenmayer (utilizando F para avanzar, +/- para las orientaciones y los corchetes [] para almacenar los estados de ramificación en una pila) , pero los colocamos dentro de las reglas de producción de una Gramática Libre de Contexto tradicional. De este modo, la red vascular se define como una estructura estática y jerárquica que el sistema puede analizar y validar de forma lineal, secuencial y determinista

Elección de Parser Descendente

En relación a lo anterior, las redes vasculares reales o simuladas presentan una anatomía compleja y repetitiva. Una rama contiene tejido y, a su vez, puede dar origen a dos sub-ramas hijas que repiten exactamente el mismo patrón geométrico (bifurcaciones binarias fractales). Esto nos planteaba dos desafíos: el análisis debía reflejar la jerarquía biológica y el algoritmo debía ser capaz de procesar cadenas con niveles de profundidad totalmente desconocidos de antemano.

El problema nos llevó hacia la elección de un Parser Descendente. Este enfoque es perfecto para el contexto médico elegido, ya que construye el Árbol Sintáctico Abstracto yendo de lo "general" a lo "particular": comienza en la raíz del problema (el axioma inicial <vaso> o tronco principal) y va "bajando" hacia los elementos periféricos (las hojas de tejido).

Para resolver la incertidumbre de la profundidad fractal, aplicamos Recursividad. En el diseño del parser, cada componente no-terminal de la gramática se transformó en un método de código. Así, la función `parse_rama()` consume sus elementos locales y, al detectar un corchete de apertura ([), se llama a sí misma de forma recursiva para procesar la sub-rama. Esto nos permitió procesar infinitos niveles de ramificación con un código simple como eficiente.

Espacios en Blanco

Para que el sistema fuera usable y legible, era indispensable que el usuario pudiera formatear las cadenas de texto largas usando espacios, tabulaciones o saltos de línea para visualizar las jerarquías de las ramas. El dilema teórico era: si el lenguaje formalmente no acepta espacios en blanco (porque no pertenecen al alfabeto), ¿en qué etapa del compilador debíamos deshacernos de ellos sin romper el diseño sintáctico?

Tomamos la decisión de filtrar los espacios en blanco estrictamente del lado del Lexer (Analizador Léxico) y mantener al Parser completamente ajeno a ellos. Esta separación de responsabilidades evitó la "contaminación" de la gramática. Si hubiéramos intentado resolverlo en el Parser, habríamos tenido que añadir elementos opcionales de espacio en cada rincón de nuestras reglas de producción (por ejemplo: $\langle \text{tronco} \rangle ::= \text{"F"} [\text{espacio}] \text{"F"} [\text{espacio}] \text{"F"}),$ destruyendo la simplicidad y la propiedad LL(1) del diseño. Al delegarlo al Lexer este consume y descarta los caracteres vacíos durante la tokenización, entregándole al Parser una secuencia de tokens completamente limpia y con significado dentro del contexto del alfabeto que esté enfocada únicamente en la estructura jerárquica pura.